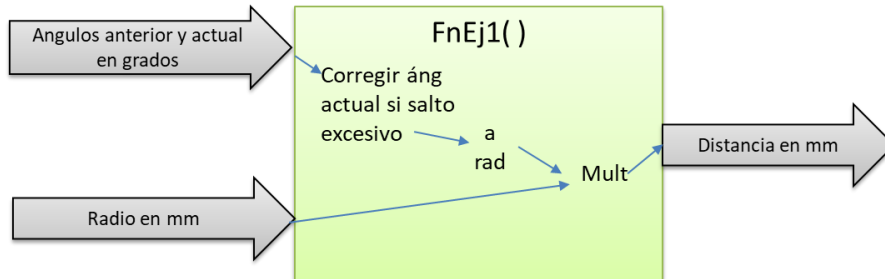


Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

- 1) *1 punto* Escribir una función que, dados el valor actual y anterior de una medición de ángulo de giro de una polea en grados (utilizar una tabla), y el radio de la polea, devuelve la distancia lineal equivalente al ángulo girado, corrigiendo el paso de 180° si es necesario:



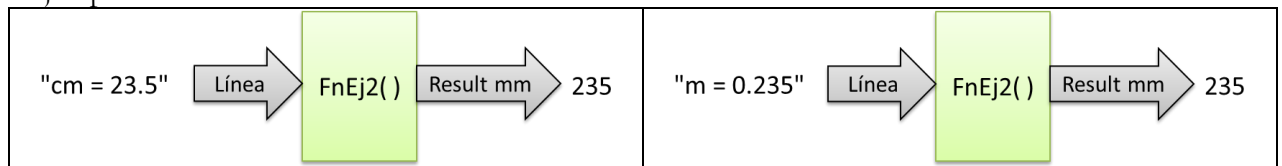
Ejemplo:

Ang anterior: 175°, Ang actual: -178° → Nuevo ang actual: 182°

Ang anterior: -172°, Ang actual: 175° → Nuevo ang actual: -185°

- 2) *1 punto* Realizar una función que, dado un texto que indica las unidades de medida y un valor, devuelva el dato real que se encuentra tras el '=' en el texto, convertido a mm. Se devolverá un valor negativo si el formato no es correcto. Las unidades de medida pueden ser mm, cm, inch o m. (1 inch=25.4mm).

Ejemplos:



(Sugerencia) Si se usan las declaraciones siguientes:

```
char* unid[]={"mm","cm","inch","m"};  
float mm_en_unid[]={1.0,10.0,25.4,1000 };
```

se puede realizar un bucle para las 4 unidades posibles donde:

unid[i] es la cadena "mm", "cm", etc. (a comparar con el comienzo de Línea)

mm_en_unid[i] es el valor 1.0, 10.0, etc. a multiplicar por el valor tras el '=' en Línea

Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

- 3) Realizar una función que, dado el estado de los bits de estado anterior y actual de una entrada digital (16 bit), devuelva:
- El índice del 'bit de peso más bajo que ha cambiado de 0 a 1', si éste está entre 0 y 7
 - El índice del 'bit de peso más bajo que ha cambiado de 0 a 1' menos 8, si el índice es superior a 7
 - Un valor negativo, si ningún bit ha cambiado de 0 a 1.

1 punto

Ejemplos de resultados según el estado de los bits de entrada:

Bit →	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Antes: →	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0
Ahora: →	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0
	Resultado: 6															
Antes: →	0	0	0	1	0	0	0	0	0	1	0	0	0	0	1	0
Ahora: →	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0
	Resultado: 6=14-8															
Antes: →	1	1	0	1	0	0	0	0	0	1	0	0	0	0	1	0
Ahora: →	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0
	Resultado: -1															

(Sugerencia)

Realizar bucle desde $iBit = 0$ hasta 15. En cada pasada:

Crear máscara para chequear bit de peso $iBit$

Comprobar bit de peso $iBit$ actual y anterior

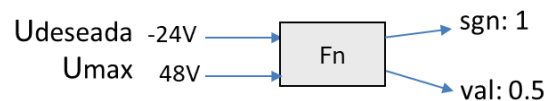
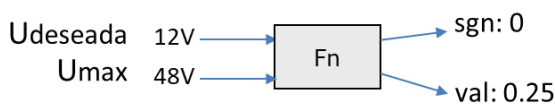
Si ha cambiado a 1 → retornar resultado $iBit$ ó $iBit-8$ según valor de $iBit$

Si el bucle llega a su fin, retornar -1

- 4) Realizar una función que, dado el valor de una tensión a obtener, y su valor absoluto máximo, devuelva los dos valores siguientes:
- Un valor entero booleano que sea verdadero si la tensión es negativa.
 - Un número real entre 0 y 1 (duty PWM) con el valor absoluto de la tensión de entrada dividido entre el máximo. Si el resultado es mayor que 1, se devolverá 1; si es menor que -1, se devolverá -1.

1 punto

Ejemplos:



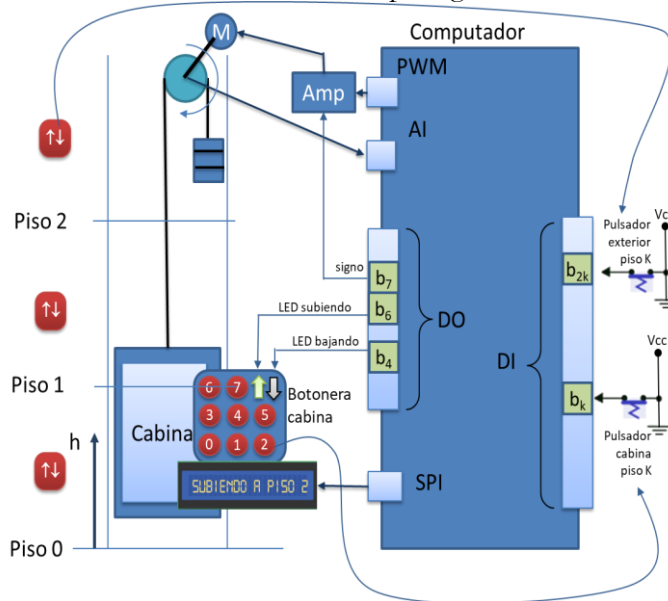
Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

5) Se desea realizar el control para gestionar un ascensor.

5 puntos

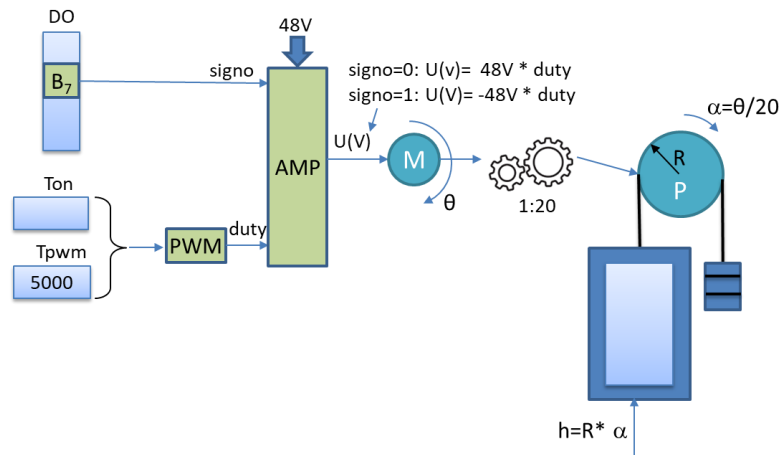


El ascensor dispone de un motor DC, que acciona una polea, en la que se ubica un sensor potenciométrico para medir el giro (y a través de él, la altura h de la cabina).

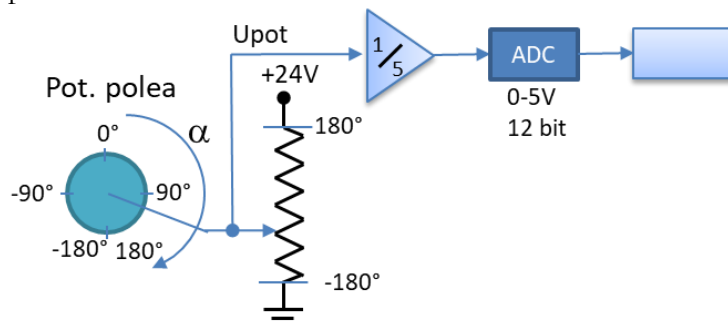
Los usuarios disponen de botones pulsadores para los pisos 0 a 7, tanto dentro de la cabina como en cada piso, que activan las entradas digitales del computador según se indica en la figura: para el piso k , el pulsador interior activa el bit de peso k de las entradas digitales (DI), y el exterior activa el bit $2*k$.

La cabina dispone de dos LEDs para indicar el sentido del movimiento, y un display LCD para indicar informaciones relevantes al usuario.

El movimiento se realiza con un motor DC de 48V, que acciona con una relación de transmisión 1:20 una polea (este valor no es relevante para el programa). Para la activación del motor se utiliza una PWM con la cadena de accionamiento siguiente, en la que un bit de las salidas digitales indicará el signo, y el duty de la PWM indica el valor absoluto. El giro positivo hace subir el ascensor.



Para el cálculo de posición h del ascensor se utiliza la cadena de medición siguiente:



Se supone que, al inicio, el ascensor está en el piso 0 con el potenciómetro indicando 0°.

Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

Se considera que el sistema tiene 3 **estados** posibles:

- ❖ *MANTENIMIENTO*: no se hace caso a ningún botón, el motor debe estar parado
- ❖ *PARADO*: sin movimiento, pero se hace caso a los botones
- ❖ *MOVIMIENTO*: en movimiento como consecuencia de alguna pulsación, se mantiene hasta la llegada al piso deseado.

El comportamiento deseado es el siguiente:

- ❖ Al inicio del programa, se piden por orden 3 líneas de texto para obtener los parámetros del ascensor, usando la fn del ejercicio 2:
 - Introduzca diámetro polea con unidades: (*ejemplo de respuesta usuario*) mm= 400
 - Introduzca altura de cada piso: (*ejemplo de respuesta usuario*) m= 3.10
 - Introduzca margen de posición: (*ejemplo de respuesta usuario*) cm= 3

Se considera que se ha llegado a un piso si:

$$(h_{\text{piso}} - \text{margen} \leq h_{\text{actual}} \leq h_{\text{piso}} + \text{margen})$$

- ❖ Se inicializa el valor de altura actual a 0mm.
- ❖ Se inicializa una $R(z)$ para control de posición:

$$R(z) = \frac{U(z)}{E(z)} = \frac{2.5 + 4.2z^{-1}}{1 - 2.5z^{-1} + 3z^{-2}}$$

Las unidades de $E(z)$ son mm de altura, y las de $U(z)$ son V a aplicar al motor.

- ❖ A continuación, se lanza una función temporizada para el control cada 25ms
- ❖ El programa principal se queda en un bucle a la espera de recibir órdenes por teclado, que dará el operario instalador: "GO" (pasa a estado *PARADO*) / "STOP" (pasa a estado *MANTENIMIENTO*).

En la rutina de control temporizada, en función del estado actual:

- ❖ *MANTENIMIENTO*: asegurar la detención del motor, apagar LEDs, indicar "MANTENIMIENTO" en LCD.
- ❖ *PARADO*: se detiene el motor, se apagan ambos LEDs, se indica "PARADO" en el LCD, y se comprueba si hay una pulsación (flanco de subida) de algún botón, para determinar cuál es el piso destino (usar fn ejercicio 3). Si hay algún destino pulsado, determinar la altura h_{piso} a alcanzar, y si es distinta a la actual, pasar a estado *SUBIENDO* ó *BAJANDO*, activar el LED correspondiente, e indicar acción y piso en display LCD ("SUBIENDO A PISO xxx").
- ❖ *MOVIMIENTO*: se desplazan las tablas temporales (e_k, u_k, α_k), se mide la altura actual a partir del ángulo girado (fn. ej 1) y se calcula el error de altura actual. A continuación, se utiliza el $R(z)$ para el cálculo de la u_k actual que permita el control de la posición. Si se ha llegado al piso deseado (ver criterio margen posición más arriba), se regresa al estado *PARADO*.

La tensión u_k resultado se aplica al motor utilizando el resultado de la fn del ejercicio 4.

Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

Librería auxiliar para E/S específica del computador destino:

```
int LeerCanalAD(); // Obtiene valor de canal A/D, 12 bits 0/5V
void InitTemporizador(int tm_ms,void (*FnISR)()); // Arranca temporizador
int ReadDI();      // Lee entrada digital
int ReadDO();      // Lee salida digital
void WriteDO(int v); // Actualiza salida digital
void LCD_printf(int col,int row,const char* txt,...); // Escribe en LCD
void InitPWM();    // Inicializa PWM
void WritePWM(float duty); // Actualiza PWM
```

Algunas funciones de C:

```
int atoi(const char* cad);           // Devuelve entero equivalente a cadena
double atof(const char* cad);        // Devuelve real equivalente a cadena
double strtod(const char* cad,char** next); // Id. a atof() y guarda en next puntero
                                         // a final de conversión
int strlen(const char* cadena);       // Devuelve longitud de cadena
char* strcpy(char* dst,const char* src); // Copia cadena fuente en destino
char* strncpy(char* dst,const char* src,int n); // Id. Máximo n caracteres
char* strcat(char* dst,const char* src); // Concatena cadena Fuente a destino
char* strncat(char* dst,const char* src,int n); // Id. Máximo n caracteres
char* strchr(const char* cad,char c); // Busca caracter en cadena, devuelve puntero
                                         // a la primera ocurrencia o NULL si no está
char* strstr(const char* cad,const char* busca); // Id. buscando cadena
int strcmp(const char* c1,const char* c2); // Compara cadenas, devuelve 0 si iguales
int strncmp(const char* c1,const char* c2,int n); // Compara cadenas hasta max n
                                         // caract, devuelve 0 si iguales
char* gets(char* destino);           // Lee cadena de consola, almacena en destino
```

Máster en Ingeniería Mecatrónica

Computadores y Programación

Examen Extraordinario – Junio 2024

APELLIDOS Y NOMBRE: _____

Cuestiones: (0.25 pts Acertada, -0.15 pts Fallada, 0 pts No Contestada)

a) ¿Cuál de estas afirmaciones es **falsa** respecto a una salida PWM ?

- ☐ El valor instantáneo varía automáticamente entre 0 y 1 a impulsos de un reloj
- ☐ Nunca puede utilizarse para LED
- ☐ El valor promedio depende de la relación entre T_{on} y T_{total} del ciclo
- ☐ Se debe amplificar según la potencia necesaria para el accionamiento al que se conecte

b) Si en el archivo .pro de Qt-Creator se añade una línea:

INCLUDEPATH += C:/User/Examen/Libreria

su efecto será:

- ☐ Permite visualizar el contenido del directorio indicado
- ☐ Añade al proyecto todos los archivos que se encuentren en el directorio indicado
- ☐ El depurador busca archivos .h para #include solamente en el directorio indicado
- ☐ El compilador busca archivos .h para #include en el directorio del proyecto y también en el directorio indicado

c) Si se desea que la función del ejercicio 1 pueda utilizar como argumento una tabla con asignación dinámica de memoria, ¿qué se debe cambiar en dicha función?

- ☐ Nada
- ☐ La función FnEj1() debe asignar la memoria con malloc() y liberar con free()
- ☐ No puede ser la misma función para tablas dinámicas que para tablas estáticas
- ☐ Las dos anteriores son ciertas

d) Completar el código de la derecha para realizar la misma funcionalidad (multiplicar complejos) que el código de la izquierda, pero usando struct:

<pre>int main() { float re_a,im_a,re_b,im_b,re_c,im_c; ... re_c=re_a*re_b-im_a*im_b; im_c=re_a*im_b+im_a*re_b; ... }</pre>	<pre>struct complejo { float re,im; }; int main() { struct complejo a,b,c; ... ¿ ? ... }</pre>
--	--

- ☐ $re.c=re.a*re.b-im.a*im.b;$ $im.c=re.a*im.b+im.a*re.b;$
- ☐ $c.re=a.re*b.re-a.im*b.im;$ $c.im=a.re*b.im+a.im*b.re;$
- ☐ $a=struct\ complejo(b)*struct\ complejo(c);$
- ☐ No es posible realizar este código con struct